

HW5 DCIC B1121123 巫煊歲

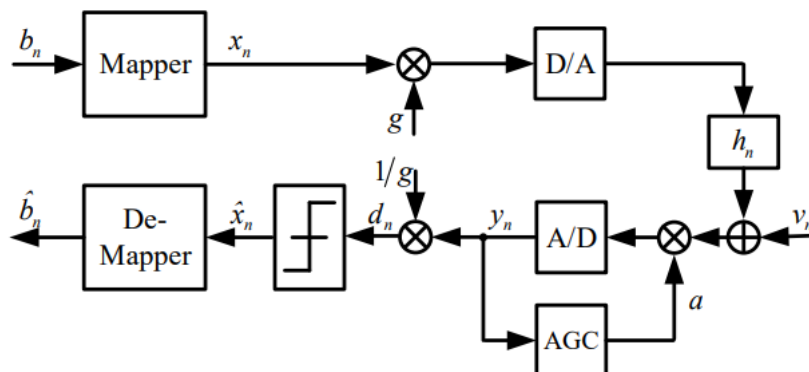


Fig. 1: Simple transceiver including a mapper, a channel, a de-mapper, a slicer, and an AGC.

首先產生一串隨機 bits(我沒有用 b.dat，因為 bit 數太少，AGC 可能無法收斂)，並透過 Mapper 依照 Gray Code 將每 4 個 bits 對應成一個 16-QAM 星座點，得到複數訊號 x_n 。接著乘上 g ，使平均能量為 1。

訊號經過通道後會受到衰減與雜訊影響，乘上通道增益 (h_n) 使振幅縮小 並加入 AWGN，形成接收訊號 (r_n)。為了補償通道造成的振幅變化，利用 AGC 乘上補償係數 $a(n)$ ，得到振幅恢復後的訊號 (y_n)。

接著再除以原先的 g ，將星座點還原得到 (d_n)。最後，Slicer 根據實部與虛部的位置進行 decision，將每個接收點映射到最接近的星座點，輸出估測符號。

程式碼

```
clear; clc; close all;

numSymbols = 1000;
N = 16;
g = 1/sqrt(10);
h_n = 0.5;
SNR_dB = -25;
mu = 0.05;

V_ref = (4*sqrt(2) + 8*sqrt(10) + 4*sqrt(18))/16 * g;

bits = randi([0 1], 1, numSymbols * 4);
bits_resaped = reshape(bits, 4, [])';
xn = zeros(1, numSymbols);

for n = 1:numSymbols
    b_real = bits_resaped(n, 1:2);
    b_imag = bits_resaped(n, 3:4);

    % Real part
    if isequal(b_real, [0 0]), re = -3;
    elseif isequal(b_real, [0 1]), re = -1;
    elseif isequal(b_real, [1 1]), re = 1;
    else, re = 3; end

    % Imaginary part
    if isequal(b_imag, [0 0]), im = -3;
    elseif isequal(b_imag, [0 1]), im = -1;
    elseif isequal(b_imag, [1 1]), im = 1;
    else, im = 3; end

    xn(n) = (re + 1i*im);
end
xn_scaled = xn * g;

sigma_v = sqrt(10^(SNR_dB/10));
```

```

noise = sigma_v * (randn(1, numSymbols) + 1i*randn(1, numSymbols)) /
sqrt(2);

rn = xn_scaled * h_n + noise;

% AGC
yn = zeros(1, numSymbols);
a = zeros(1, numSymbols);
a(1) = 1;

for n = 1:numSymbols-1
    yn(n) = a(n) * rn(n);

    % Moving Sum / N
    if n >= N
        curr_avg_amp = mean(abs(yn(n-N+1:n)));
    else
        curr_avg_amp = mean(abs(yn(1:n)));
    end

    a(n+1) = a(n) + mu * (V_ref - curr_avg_amp);
end
yn(numSymbols) = a(numSymbols) * rn(numSymbols);

dn = yn / g;

f_slice = @(v) (3*(v >= 2) + 1*(v >= 0 & v < 2) + ...
               -1*(v >= -2 & v < 0) + -3*(v < -2));
x_hat = f_slice(real(dn)) + 1i*f_slice(imag(dn));

figure;
plot(a, 'LineWidth', 1.5);
title('AGC 增益 (a) 收敛曲线 mu = 0.05');
xlabel('Symbol Index'); ylabel('Gain a');
grid on;

```

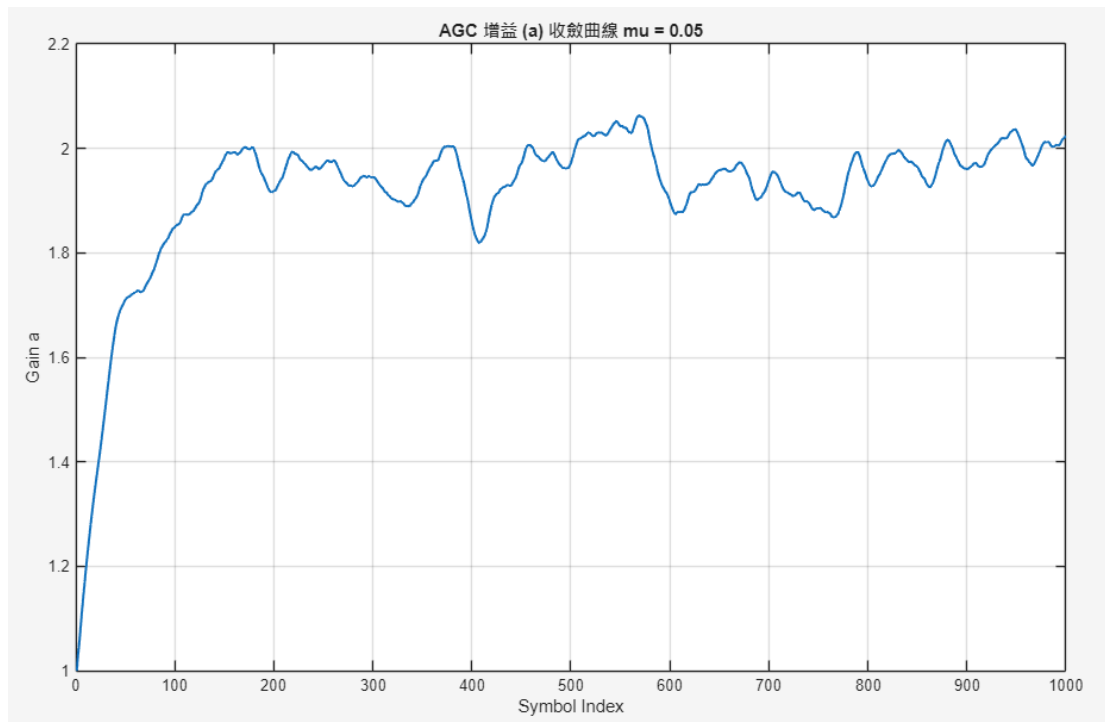
```

figure;
plot(real(dn(1:100)), imag(dn(1:100)), 'r. ');
hold on;
plot([-3 -1 1 3], [1 1 1 1], 'k+', 'MarkerSize', 10);
plot([-3 -1 1 3], [3 3 3 3], 'k+', 'MarkerSize', 10);
plot([-3 -1 1 3], [-1 -1 -1 -1], 'k+', 'MarkerSize', 10);
plot([-3 -1 1 3], [-3 -3 -3 -3], 'k+', 'MarkerSize', 10);
title('收斂前星座圖 (前面 100 點)');
xlabel('In-phase'); ylabel('Quadrature');
axis([-5 5 -5 5]); grid on;

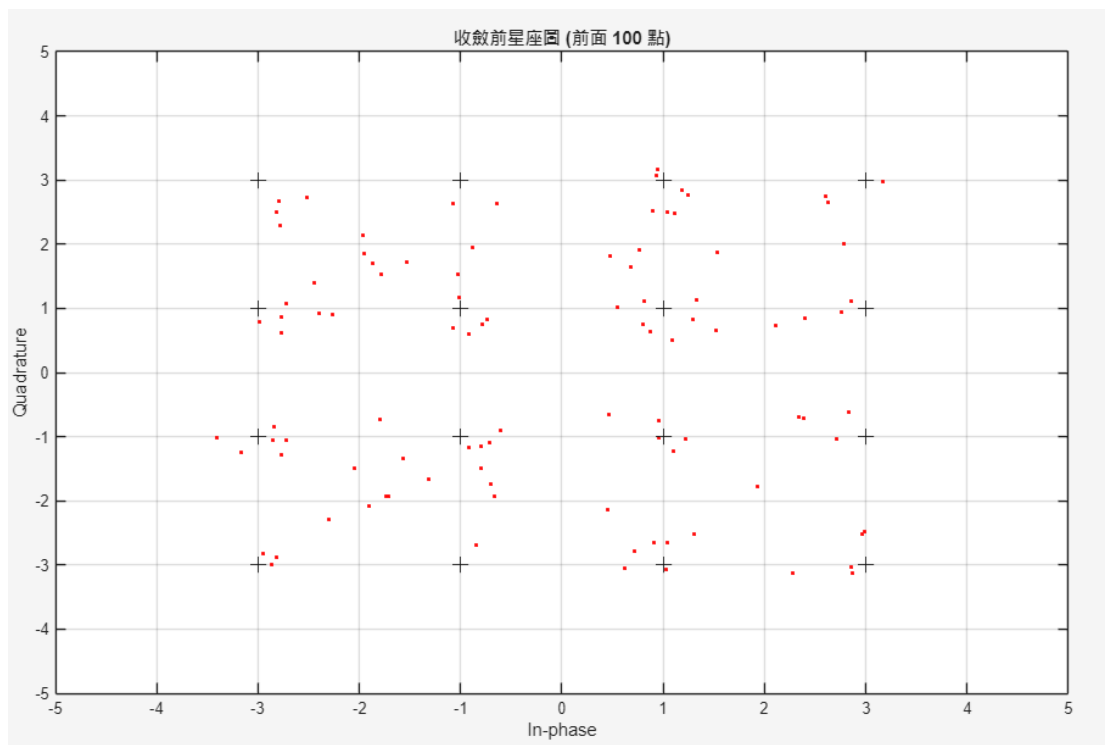
figure;
plot(real(dn(end-100:end)), imag(dn(end-100:end)), 'r. ');
hold on;
plot([-3 -1 1 3], [1 1 1 1], 'k+', 'MarkerSize', 10);
plot([-3 -1 1 3], [3 3 3 3], 'k+', 'MarkerSize', 10);
plot([-3 -1 1 3], [-1 -1 -1 -1], 'k+', 'MarkerSize', 10);
plot([-3 -1 1 3], [-3 -3 -3 -3], 'k+', 'MarkerSize', 10);
title('收斂後星座圖 (最後 100 點)');
xlabel('In-phase'); ylabel('Quadrature');
axis([-5 5 -5 5]); grid on;

```

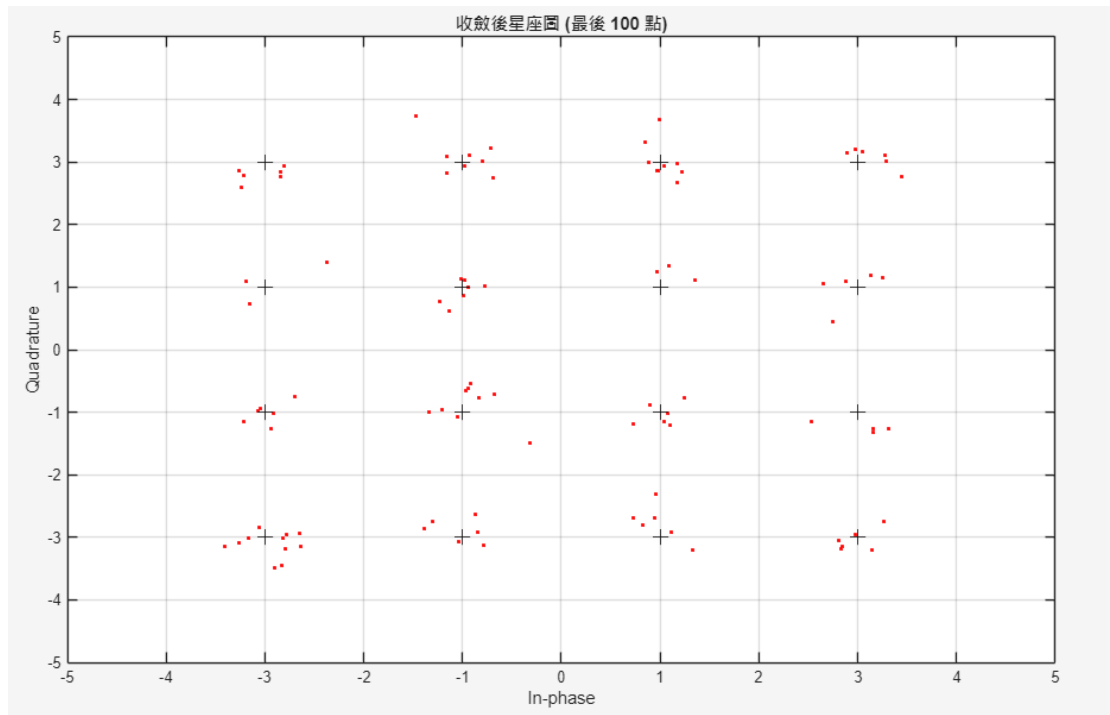
結果圖



觀察 AGC 從初始值 1 收斂到 2 的過程。



畫出前面 100 個 symbol 的星座圖，紅點是實際接收點，黑色 + 是理想星座圖位置。可以看到收斂前紅點散落在 16 個星座點。



畫出後面 100 個 symbol 的星座圖，可以看到收斂後紅點聚集在 16 個星座點附近。

說明

`numSymbols = 5000;` 模擬用的符號總數，越多結果越穩定

`N = 16;` AGC 移動平均的視窗長度，決定平均幾個符號來估算振幅

`g = 1/sqrt(10);` 讓 16-QAM 符號的平均功率 normalize 為 1

`h_n = 0.5;` 模擬訊號經過通道後振幅縮小為 0.5 倍

`SNR_dB = 25;`

`mu = 0.05;` 越大收斂越快但可能不穩定

$V_{ref} = (4\sqrt{2}) + 8\sqrt{10} + 4\sqrt{18}) / 16 * g;$

16-QAM 共 16 個星座點，座標為 $(\pm 1, \pm 1)$, $(\pm 1, \pm 3)$, $(\pm 3, \pm 1)$, $(\pm 3, \pm 3)$

各點到原點的距離（振幅）：

- $(\pm 1, \pm 1) \rightarrow \sqrt{2}$ ，共 4 點
- $(\pm 1, \pm 3)$ 或 $(\pm 3, \pm 1) \rightarrow \sqrt{10}$ ，共 8 點

- $(\pm 3, \pm 3) \rightarrow \sqrt{18}$ ，共 4 點

平均振幅 = $(4\sqrt{2} + 8\sqrt{10} + 4\sqrt{18}) / 16$ ，再乘上 g ，這個值是 AGC 的收斂目標。

```
bits = randi([0 1], 1, numSymbols * 4);
```

```
bits_resaped = reshape(bits, 4, [])';
```

隨機產生 $\text{numSymbols} \times 4$ 個 bit，每 4 個 bit 對應一個 16-QAM 符號，

reshape 後每列是一個 symbol (4 個 bit)。

bits_resaped x					
1000x4 double					
	1	2	3	4	
1	1	0	1	1	
2	0	0	1	0	
3	1	0	1	0	
4	1	1	0	0	
5	0	1	1	0	
6	1	0	1	1	
7	0	0	1	1	
8	1	0	0	0	
9	0	1	0	0	
10	1	0	0	1	
11	1	1	1	0	
12	1	1	0	1	
13	1	0	0	1	
14	1	1	1	0	
15	0	1	1	0	
16	1	0	1	0	
17	1	0	0	1	

```
b_real = bits_resaped(n, 1:2); % 前 2 bit 決定實部
```

```
b_imag = bits_resaped(n, 3:4); % 後 2 bit 決定虛部
```

```
rn = xn_scaled * h_n + noise;
```

接收訊號 = 傳送訊號 $\times$ 通道係數 (0.5 倍衰減) + 雜訊。

AGC 的目的是補償通道衰減，讓接收到的訊號振幅自動回到正常水準。

$a(1) = 1$; 初始增益設為 1

$y(n) = a(n) * r(n)$; 將接收訊號乘上增益，得到 AGC 輸出。

if $n \geq N$

$\text{curr_avg_amp} = \text{mean}(\text{abs}(y(n-N+1:n)))$;

else

$\text{curr_avg_amp} = \text{mean}(\text{abs}(y(1:n)))$;

end

N 個輸出的 moving avg (不足 N 個時用當前全部)，估計目前輸出的強度。

$a(n+1) = a(n) + \mu * (V_{\text{ref}} - \text{curr_avg_amp})$;

$V_{\text{ref}} - \text{curr_avg_amp}$ = 誤差 (目標振幅 - 實際振幅)

誤差 > 0 (振幅太小) $\rightarrow$ 增益增加

誤差 < 0 (振幅太大) $\rightarrow$ 增益減少

μ 控制每次調整的幅度